

CPAP Therapy Pressure Control — Block Diagram Reference

Document: Pressure control as-built (code reference)

Firmware repo: CPAP_InitialCode

Primary sources: CM7/Core/Src/main.c, blower_ctrl.c, tube_comp.c, flow_sensor.c, leak_estimator.c, therapy.c

Version: 1.0

Date: 2025-06-25

Print / PDF: Open this file in VS Code, Cursor, or GitHub and use **Print to PDF**, or export with Pandoc. Mermaid diagrams render in GitHub and many Markdown previewers; for offline PDF with diagrams, use a Mermaid-capable exporter (e.g. mermaid.live paste-export).

Table of contents

1. Executive summary
 2. Task architecture and rates
 3. Sensor measurement chain
 4. Therapy setpoint path
 5. Outer pressure controller (BlowerCtrl_Step)
 6. Feed-forward blocks
 7. Closed-loop plant view
 8. CM4 actuator path
 9. Safety and parallel logic
 10. USB telemetry mapping
 11. Debugger watch symbols
 12. Mode exceptions
-

1. Executive summary

Therapy pressure control runs on **CM7** in **SensorTask** at **200 Hz** (5 ms period). Each tick:

1. Harvest SPI sensor samples (flow @ 200 Hz, pressure @ ~190 Hz).
2. Convert flow differential pressure → **total flow** Q_{total} .
3. Estimate **mask pressure** P_{mask} from P_{sensor} and Q_{total} (tube compensation).
4. Update **leak estimator** → **patient flow** $Q_{patient}$.
5. Run **outer PID + feed-forward** → post **RPM command** to CM4.
6. Run **therapy state machine** (ramp, EPR, APAP, mask-off).

The PID **feedback** is the **estimated mask pressure** (`BlowerCtrl_GetMeasured()`), not the raw blower sensor, when tube compensation is enabled. Feed-forward terms (steady-state, flow, dip) add proactive RPM on top of the PID trim.

2. Task architecture and rates

2.1 RTOS tasks (pressure-relevant)

```

flowchart TB
    subgraph CM7["CM7"]
        direction TB
        subgraph ST["SensorTask - osPriorityNormal - 5 ms"]
            A["1. SPI harvest FS / PS / AS"]
            B["2. Flow_Update"]
            C["3. Leak_Update"]
            D["4. BlowerCtrl_Step"]
            E["5. Therapy_Update"]
            F["6. Telem_PushSample @ 50 Hz"]
            A --> B --> C --> D --> E --> F
        end
        subgraph SAFT["SafetyTask - osPriorityRealtime - 5 ms"]
            G["Safety_Supervisor_Update"]
        end
    end

    subgraph CM4["CM4"]
        H["IPC poll in main while(1)"]
        I["FOC speed loop ~1000 Hz"]
        J["Blower motor"]
        H --> I --> J
    end

    D -->|"BlowerIpc_CM7_SetSpeedRpmSilent"| H
    G -. ->|"fault → Therapy_Stop"| E

```

2.2 Sample and control rates

Block	Rate	Notes
SensorTask loop	200 Hz	period_ms = 5 in main.c
Flow sensor AMS5935 (FS)	200 Hz	SPI2, every tick
Pressure sensor AMS5935 (PS)	~190 Hz	SPI4; 1/20 ticks used for AS

Block	Rate	Notes
Atmospheric AMS5935 (AS)	10 Hz	AS_DECIMATE = 20
CM4 IPC status read	100 Hz	IPC_DECIMATE = 2
BlowerCtrl_Step	200 Hz	Same as SensorTask
RPM command to CM4	200 Hz	SetSpeedRpmSilent, ramp_ms = 0
USB telemetry T,...	50 Hz	tick_n % 4 == 0
CM4 FOC inner loop	~1000 Hz	Motor speed regulation

2.3 SensorTask tick order (code order)

```

Harvest FS → Harvest PS or AS → Start next conversions
  → Leak_Update(Q_total, P_mask_est)
  → BlowerCtrl_Step(dt)
  → Therapy_Update(dt)
  → Telem_PushSample (every 4th tick)

```

3. Sensor measurement chain

3.1 Hardware → software signals

flowchart LR

```

    subgraph HW["Physical sensors"]
        FS["AMS5935 FS<br/>dP across flow element"]
        PS["AMS5935 PS<br/>gauge pressure at blower"]
        AS["AMS5935 AS<br/>barometric"]
    end

    subgraph FLOW["flow_sensor.c"]
        LPF["LPF on dP<br/>g_flow_lpf_cutoff_hz"]
        ZO["Zero offset"]
        CAL["Dp → Q LUT"]
        QT["Q_total<br/>Flow_GetSlm()"]
        FS --> LPF --> ZO --> CAL --> QT
    end

    subgraph PRESS["Pressure path"]
        PSMB["P_sensor cmH2O<br/>measPS → MBAR_TO_CMH20"]
        PS --> PSMB
    end

    subgraph TUBE["tube_comp.c"]
        DROP["dP_hose = R_lam·|Q| + R_turb·Q·|Q|"]
    end

```

```

    MASK["P_mask = P_sensor - sign(Q) · |dP_hose|"]
    QT --> DROP
    PSMB --> DROP --> MASK
end

subgraph LEAK["leak_estimator.c"]
    QV["Q_vent = K_vent · √P_mask_filt"]
    QLK["Q_leak slow LPF 10 s"]
    QP["Q_patient = Q_total - Q_vent - Q_leak"]
    MASK --> QV
    QT --> QLK
    QT --> QP
    QV --> QP
    QLK --> QP
end

```

3.2 Signal definitions

Symbol	API	Definition
P_sensor	BlowerCtrl_GetSensorAMS2035	Sensor AMS2035 PS at device (cmH ₂ O)
Q_total	Flow_GetSlm()	Total hose flow from FS (slm)
P_mask	BlowerCtrl_GetMasked()	Estimated compensated mask estimate (cmH ₂ O)
Q_patient	Flow_GetPatientSlm()	Flow from FS – vent – unintentional leak (filtered)
Q_patient_raw	Flow_GetPatientRawSlm()	Unfiltered patient residual (for fast FF)

3.3 Tube compensation model

$$dP_{hose}(Q) = R_{lam} \cdot |Q| + R_{turb} \cdot Q \cdot |Q| \quad [\text{cmH}_2\text{O}]$$

$$P_{mask} = P_{sensor} - \text{sign}(Q) \cdot |dP_{hose}|$$

- $Q > 0$ (inspiration toward patient): $P_{mask} < P_{sensor}$
- $Q < 0$ (expiration): $P_{mask} > P_{sensor}$
- Defaults: $g_{tubecomp_enabled} = 1$, $R_{turb} = 0.00015$, $R_{lam} = 0$
- If disabled: $P_{mask} = P_{sensor}$ (pass-through)

3.4 What uses which signal

Consumer	Pressure	Flow
PID feedback	P_mask (est.)	Q_total (tube comp only)

Consumer	Pressure	Flow
Flow FF	—	Q_patient (max raw/filt in inhale dir.)
Dip FF	P_sensor drop rate + P_mask sag	inhale gate via Q_patient
Leak / mask-off	P_mask	Q_total
EPR breath detect	P_mask	Q_patient
Safety supervisor	P_sensor	RPM, setpoint band
USB plot P_sensor	raw sensor	—
USB plot P_mask	mask estimate	—

4. Therapy setpoint path

Therapy_Update() runs **after** BlowerCtrl_Step() each tick but setpoint changes apply on the **next** tick via BlowerCtrl_SetSetpoint().

flowchart TB

```

USB["USB / GUI therapy commands"] --> SM["therapy.c state machine"]
SM --> IDLE["IDLE"]
SM --> RAMP["RAMP: 4 → target cmH2O"]
SM --> RUN["RUNNING"]
SM --> STOP["STOPPING → IDLE"]

```

```

RUN --> MODE{"Mode?"}

```

```

MODE -->|CPAP| FIX["Hold g_therapy_target_cmh2o"]

```

```

MODE -->|APAP| APAP["Apap_Update() adjusts target<br/>may enable BlowerOsc for FOT"]

```

```

FIX --> EPR["Epr_ModifySetpoint()"]

```

```

APAP --> EPR

```

```

RAMP --> EPR

```

```

EPR --> BSP["BlowerCtrl_SetSetpoint(user SP)"]

```

```

BSP --> SLEW["Internal slew limiter<br/>g_blower_ctrl_setpoint_slew"]

```

```

START["Therapy_Start"] --> EN["BlowerCtrl_SetEnable(true)"]

```

```

STOPCMD["Therapy_Stop / fault"] --> DIS["BlowerCtrl_SetEnable(false)"]

```

4.1 Effective PID setpoint (inside BlowerCtrl_Step)

```

setpoint_eff = g_blower_ctrl_setpoint_slew + BlowerOsc_sample

```

- **Slew limiter** ramps user setpoint at `g_blower_ctrl_slew_cmh2o_per_s` (separate up-rate available).
- **BlowerOsc** adds sinusoidal perturbation (default off; used for APAP/FOT).
- **Gain scheduling** uses **slewed** setpoint (without oscillation) to pick LOW / MID / HIGH K_p , K_i , K_d .

5. Outer pressure controller (BlowerCtrl_Step)

5.1 Block diagram

```

flowchart TB
    subgraph MEAS["Measurement"]
        PS["P_sensor"] --> TC["TubeComp_Apply(Q_total)"]
        TC --> RAW["P_mask raw"]
        RAW --> MF["Meas LPF = g_blower_ctrl_meas_filter_alpha"]
        MF --> FM["Fast meas enabled?<br/>inhaling sag > thresh"]
        FM -->|yes| BL["Blend filt raw P_mask"]
        FM -->|no| MOUT["meas_cmh2o"]
        BL --> MOUT
    end

    subgraph SP["Setpoint"]
        USP["User SP (slew-limited)"] --> ZN["Zone select → Kp, Ki, Kd"]
        USP --> OSC["+ BlowerOsc"]
        OSC --> SPE["setpoint_eff"]
    end

    subgraph PID["PID trim (RPM)"]
        ERR["error = setpoint_eff - meas_cmh2o"]
        MOUT --> ERR
        SPE --> ERR
        ZN --> PT["P = Kp·error<br/>(× over_P_gain if error < 0)"]
        ZN --> IT["I += Ki·error·dt<br/>conditional + anti-windup"]
        ZN --> DT["D = -Kd·d(meas)/dt<br/>filtered"]
        ERR --> PT
        ERR --> IT
        MOUT --> DT
    end

    subgraph FF["Feed-forward (RPM)"]
        FF0["FF = K·√SP + offset"]
        FFL["Flow FF (slew-limited)"]
        FFD["Dip FF (slew-limited)"]
    end

```

```

subgraph SUM["Output"]
  ADD["rpm_unfilt = FF + Flow_FF + Dip_FF + P + I + D"]
  FFO --> ADD
  FFL --> ADD
  FFD --> ADD
  PT --> ADD
  IT --> ADD
  DT --> ADD
  ADD --> SAT["Clamp rpm_min ... rpm_max"]
  SAT --> OF["Output LPF"]
  OF --> OUT["g_blower_ctrl_out_rpm"]
  OUT --> IPC["BlowerIpc_CM7_SetSpeedRpmSilent"]
end

```

5.2 PID term summary

Term	Equation / rule
Error	$\text{setpoint_eff} - \text{meas_cmh2o}$
P	$K_p \cdot \text{error}$; if $\text{error} < -\text{over_p_thresh} \rightarrow \times \text{over_p_gain}$
I	$K_i \cdot \text{error} \cdot dt$; frozen while setpoint still slewing up at start; conditional integration when saturated
D	$-K_d \cdot d(\text{meas})/dt$ (D-on-measurement, LPF on derivative)
Anti-windup	Reject I step if deepening saturation; clamp I to RPM headroom after FF

5.3 Gain zones (scheduled on slew setpoint)

Zone	Setpoint range (cmH ₂ O)	Typical use
LOW	4 – 8	Steep blower gain $\rightarrow$ lower K_p/K_i
MID	8 – 15	Transition
HIGH	15 – 25	Higher K_p/K_i for fine control

Hysteresis: $\text{BLOWER_CTRL_ZONE_HYST_CMH2O} = 0.5$

6. Feed-forward blocks

6.1 Steady-state FF (g_blower_ctrl_ff_enable)

$\text{FF} = \text{g_blower_ctrl_ff_k_rpm_per_sqrtcmh} \cdot \sqrt{(\text{setpoint})} + \text{g_blower_ctrl_ff_offset_rpm}$

Carries most of the DC RPM so the integrator acts as a small trim (default `integ_max` ~4 krpm with FF on).

6.2 Flow FF (`g_blower_ctrl_flow_ff_enable`)

Inputs

- `q_pat` = `inhalePatientFlowSlm()` → `max(Q_patient_raw, Q_patient_filt)` in inhale direction
- `dq/dt` from successive `q_pat` samples
- Gated by `patientInhaling()` (above deadband)

Target (positive inhale sign)

```
if q_pat > deadband:
    ff_target = k · (q_pat - deadband)
if dq/dt > onset_thresh:
    ff_target += onset_k · (dq/dt - onset_thresh)    [capped]

Slew-limited to s_ff_flow_slew_rpm using flow_ff_rise_rpm_per_s /
flow_ff_fall_rpm_per_s.
```

6.3 Dip FF (`g_blower_ctrl_dip_ff_enable`)

Anticipates pressure sag from **fast drop of P_{sensor}** during inhalation.

Gates (all required)

1. `patientInhaling()`
2. `setpoint - Pmask > dip_ff_err_min`
3. `dPsensor/dt < -dip_ff_thresh` (filtered drop rate)

```
ff_dip_target = dip_k · (-dPsensor/dt - thresh)    [capped at dip_ff_max_rpm]
```

Slew-limited separately; **zero on expiration** by design.

6.4 Combined disturbance FF

```
ff_dist = ff_flow + ff_dip
rpm_unfilt = FF + ff_dist + P + I + D
```

7. Closed-loop plant view

flowchart LR

```
SP["Therapy setpoint<br/>(mask pressure target)"] --> CTRL["BlowerCtrl<br/>CM7 outer loop"]
CTRL --> RPM["RPM cmd"]
RPM --> CM4["CM4 FOC"]
CM4 --> MOTOR["Blower"]
MOTOR --> HOSE["Hose + mask + patient"]
```



```

HOSE --> PS["P_sensor @ device"]
HOSE --> FS["Flow sensor"]
FS --> QT["Q_total"]
PS --> TC["Tube compensation"]
QT --> TC
TC --> PM["P_mask estimate"]
PM --> CTRL
QT --> LEAK["Leak estimator"]
PM --> LEAK
LEAK --> QP["Q_patient"]
QP --> CTRL

```

Important: There is no sensor at the mask. Mask pressure is **estimated**; patient flow is **derived** from total flow minus vent and leak models.

8. CM4 actuator path

sequenceDiagram

```

participant ST as SensorTask CM7
participant IPC as Shared mailbox
participant CM4 as CM4 main loop
participant FOC as FOC speed loop

```

```

ST->>IPC: SetSpeedRpmSilent(rpm_cmd) @ 200 Hz
CM4->>IPC: Poll cm7_cmd_seq
CM4->>FOC: Update speed reference
FOC->>FOC: Torque / flux control ~1 kHz
CM4->>IPC: mech_speed_rpm status
ST->>IPC: GetStatus @ 100 Hz

```

Signal	Direction	Content
cm7_cmd_seq	CM7 → CM4	New RPM command flag
target_speed_rpm	CM7 → CM4	PID output
mech_speed_rpm	CM4 → CM7	Measured motor speed (telemetry)

9. Safety and parallel logic

SafetyTask runs at 5 ms with **higher priority** than SensorTask. It does **not** compute PID; it monitors and can stop therapy.

Check	Input	Action
Sensor range	<code>P_sensor</code>	Fault if implausible
Low pressure + high RPM	<code>P_sensor</code> , <code>rpm_cmd</code>	Hose disconnect suspicion
Over-pressure band	<code>P_sensor</code> vs target	Fault if exceeded
CM4 heartbeat	IPC status age	Fault if CM4 unresponsive

Mask-off (`therapy.c`): high unintentional leak → reduced RPM hold; uses `Leak_GetInstantExcessSlm()` and `Leak_IsHigh()`.

EPR (`epr.c`): modifies expiratory setpoint; uses `Q_patient` + `P_mask` for phase.

APAP (`apap_ctrl.c`): adjusts session target; may enable `BlowerOsc` for forced-oscillation probing.

10. USB telemetry mapping

Wire format (50 Hz):

T,<ms>,<P_sensor>,<Q_patient>,<pid_rpm_cmd>,<mech_rpm>,<P_mask>

Field	Firmware source
<code>P_sensor</code>	<code>BlowerCtrl_GetSensorCmh2o()</code>
<code>Q_patient</code>	<code>Flow_GetPatientSlm()</code>
<code>pid_rpm_cmd</code>	<code>BlowerCtrl_GetOutputRpm()</code>
<code>mech_rpm</code>	<code>g_blower_status.mech_speed_rpm</code>
<code>P_mask</code>	<code>BlowerCtrl_GetMeasured()</code>

Python plotter: `pc_tools/cpap_telemetry_plot.py` — blue `P_sensor`, teal `P_mask` (est.) on same axes.

11. Debugger watch symbols

11.1 Pressure and flow

Symbol	Meaning
<code>g_blower_ctrl_sensor_cmh2o</code>	Raw <code>P_sensor</code>
<code>g_blower_ctrl_meas_cmh2o</code>	<code>P_mask</code> estimate (PID measurement)
<code>g_blower_ctrl_meas_filt_cmh2o</code>	LPF'd measurement
<code>g_blower_ctrl_error_cmh2o</code>	<code>setpoint_eff</code> - <code>meas</code>

Symbol	Meaning
<code>g_tubecomp_drop_cmh2o</code>	Computed hose drop
<code>g_tubecomp_p_mask_cmh2o</code>	Last tube-comp mask value
<code>g_flow_q_slm</code>	<code>Q_total</code>
<code>g_flow_patient_slm</code>	<code>Q_patient</code> (leak module)
<code>g_leak_uninten_slm</code>	Slow unintentional leak estimate

11.2 PID and FF terms (RPM)

Symbol	Meaning
<code>g_blower_ctrl_ff_term</code>	Steady-state FF
<code>g_blower_ctrl_ff_flow_term</code>	Flow FF (after slew)
<code>g_blower_ctrl_ff_dip_term</code>	Dip FF (after slew)
<code>g_blower_ctrl_p_term</code>	P contribution
<code>g_blower_ctrl_i_term</code>	Integrator state
<code>g_blower_ctrl_d_term</code>	D contribution
<code>g_blower_ctrl_out_rpm</code>	Final RPM command
<code>g_blower_ctrl_saturated</code>	Non-zero if clamped

11.3 Setpoint and mode

Symbol	Meaning
<code>g_blower_ctrl_setpoint_user</code>	User / therapy target
<code>g_blower_ctrl_setpoint_slew</code>	Slewed internal setpoint
<code>g_blower_ctrl_zone</code>	Active gain zone (0=LOW, 1=MID, 2=HIGH)
<code>g_blower_ctrl_enabled</code>	Master PID enable
<code>g_tubecomp_enabled</code>	Tube compensation on/off
<code>g_blower_ctrl_wm6850_mode</code>	Reference mode bypass

12. Mode exceptions

Condition	Behaviour
<code>g_tubecomp_enabled = 0</code>	<code>P_mask = P_sensor</code> ; PID regulates blower-side pressure
<code>g_blower_ctrl_wm6850_mode = 1</code>	Raw sensor PID @ 100 Hz ; FF, slew zones, tube comp, oscillation bypassed on PID path

Condition	Behaviour
<code>g_blower_ctrl_enabled = 0</code>	No RPM posts from PID; measurement tracking only
<code>g_blower_ctrl_pid_enable = 0</code> Manual bench mode	Output = FF terms only (no P/I/D) <code>BlowerCtrl_ManualStart/SetSpeed</code> bypasses pressure PID

Appendix A — One-page ASCII overview (print-friendly)

```

                                THERAPY SETPOINT (therapy.c)
CPAP target / APAP adjust   EPR   BlowerCtrl_SetSetpoint   slew

                                setpoint_eff = slew + Blower0sc

AMS5935      Flow_Update      Leak_Update      BlowerCtrl_Step
PS → P_s     FS → Q_tot       → Q_patient      P_mask = TubeComp()
                                PID(P,I,D) + FF
                                + Flow_FF + Dip_FF
                                → RPM cmd @ 200 Hz

                                CM4 FOC → Blower → Hose

```

Appendix B — Related files

File	Role
<code>CM7/Core/Src/main.c</code>	<code>SensorTask</code> , sensor scheduling, call order
<code>CM7/Core/Src/blower_ctrl.c</code>	Outer PID + all FF + RPM output
<code>CM7/Core/Src/tube_comp.c</code>	Mask pressure estimate
<code>CM7/Core/Src/flow_sensor.c</code>	FS conditioning, <code>Q_total</code>
<code>CM7/Core/Src/leak_estimator.c</code>	Vent/leak model, <code>Q_patient</code>
<code>CM7/Core/Src/therapy.c</code>	Therapy SM, setpoint, start/stop
<code>CM7/Core/Src/apap_ctrl.c</code>	APAP target + FOT oscillation
<code>CM7/Core/Src/blower_osc.c</code>	Sinusoidal setpoint perturbation
<code>CM7/Core/Src/safety_supervisor.c</code>	Fault detection
<code>CM7/Core/Src/telemetry_stream.c</code>	USB T, ... lines

File	Role
<code>pc_tools/cpap_telemetry_plot.py</code>	Live plots

End of document